Check for updates

Hindawi
Mobile Information Systems
Volume 2017, Article ID 9038634, 16 pages
https://doi.org/10.1155/2017/9038634

Research Article

A New Theoretical Approach: A Model Construct for Fault Troubleshooting in Cloud Computing

Ameen Alkasem, Hongwei Liu, Muhammad Shafiq, and Decheng Zuo

School of Computer Science and Technology, Harbin Institute of Technology, Harbin 150001, China

Correspondence should be addressed to Ameen Alkasem; 77ameen@ftcl.hit.edu.cn

Received 29 March 2017; Revised 8 August 2017; Accepted 6 September 2017; Published 19 October 2017

Academic Editor: Bartolomeo Montrucchio

Copyright © 2017 Ameen Alkasem et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

In cloud computing, there are four effective measurement criteria: (I) priority, (II) fault probability, (III) risk, and (IV) the duration of the repair action determining the efficacy of troubleshooting. In this paper, we propose a new theoretical algorithm to construct a model for fault troubleshooting; we do this by combining a Naïve-Bayes classifier (NBC) with a multivalued decision diagram (MDD) and influence diagram (ID), which structure and manage problems related to unambiguous modeling for any connection between significant entities. First, the NBC establish the fault probability based on a Naïve-Bayes probabilistic model for fault diagnosis. This approach consists of three steps: (I) identifying the network parameters to also show the reliance for probability relationship among the entire set of nodes; (II) determining the structure of the network topology; (III) assessing the probability of the fault being propagated. This calculates the probability of each node being faulty given the evidence. Second, the MDD combines the influential factors of four measurements and determines the utility value of prioritizing their actions during each step of the fault troubleshooting which in turn assesses which fault is selected for repair. We demonstrate how the procedure is adapted by our method, with the host server's failure to initiate a case-study. This approach is highly efficient and enables low-risk fault troubleshooting in the field of cloud computing.

1. Introduction

Cloud computing refers to flexible, self-service, and network-accessible computing resource pools that can be allocated to meet demand, allowing thousands of virtual machines (VMs) to be used even by small organizations. As the scale of cloud deployments continues to grow, the necessity for scalable monitoring tools that support the unique requirements of cloud computing is of greater priority [1]. Endowing clouds with fault-troubleshooting abilities for the management of the reduction, existing complexity, and continued development of the complexity of a utility cloud [2] presents a difficult but obviously interesting solution. Troubleshooting for faults in the cloud remains relatively unexplored so that there has been no universally accepted tool chain or systems for the purpose. In the presented work fault detection is defined as a problem of classifying the time instances during runtime in which cloud computing is having anomalies and faults [3]. A useful algorithm in which the problem needs to be addressed must contain the following: (a) high detection and rates with few false alarms; (b) an independently supervised technique due to insufficient a priori knowledge regarding standard or anomalous behavior, or circumstances; (c) an autonomic methodology so that the increase in cloud scaling includes personnel costs. Additionally, a cloud is typically used by multiple tenants and multiple VMs may be colocated on the same host server [4]. Because resources such as CPU usage, memory usage, and network overhead are not virtualized, there exists a complex dynamism between the VMs, which have to share resources. The increased complexity is hugely problematic in terms of troubleshooting [5].

However, it is unsuitable for environments with dynamic changes, is susceptible to high false-alarm rates, and is expected to perform poorly in the context of large-scale cloud computing [6]. Current monitoring tools are service tools, such as Nagios and Ganglia [7, 8] which are designed with greater functionality to obtain low-level metrics including the CPU, disk, memory, and I/O, but not designed to service

monitoring for dynamic cloud environments. Current apparatus are founded on threshold methods, which in industry monitoring commodities are more frequently employed. First, upper and lower bounds are created for all of the metrics. The threshold values come from the knowledge of performance along with predictions rooted in the historical data analysis from the long-term. Violation of the threshold limit by any of the observed metrics triggers an alarm of anomaly. Therefore, in this paper we propose a new theoretical approach for constructing the steps of a fault-troubleshooting model. The model combines a Naïve-Bayes classifier (NBC) with a multivalued decision diagram (MDD) and influence diagram (ID) to structure and manage fault troubleshooting. An NBC is a simple probabilistic classifier algorithm based on the application of a Bayesian network (BN) with independence assumptions between predictors [9, 10]. The NBC is a modeling technique based on probability, which is the most acceptable for knowledge-based diagnostic systems which are knowledge-based. Ultimately this allows for the modeling and reasoning regarding uncertainty. An NBC and an ID were developed in the artificial intelligence community as principled formalisms for reasoning and representing under uncertainty in intelligent systems [11]. Therefore, this technique is ideally suited for the diagnosing in real-time world complication where uncertain incomplete datasets exist [12]; this method offers an appropriate mechanism for diagnosing complicated virtualization systems in the cloud.

The major advantage of using a BN is its ability to represent and hence allow knowledge to be understood. A BN comprises two parts: qualitative knowledge through its network structure and quantitative knowledge through its parameters. Whereas expert knowledge from practitioners is mostly qualitative, it can be used directly for building the structure of a BN [13]. In addition, data mining algorithms can encode both qualitative and quantitative knowledge and are able to encode both forms of knowledge simultaneously in a BN. Therefore, BNs can bridge the gap between different types of knowledge and serve to unify all knowledge availability into a single form of representation [14]; therefore, an NBC is suitable for producing probability estimates rather than predictions. These estimates allow predictions to be ranked and their expected costs to be minimized. The NBC still provides a high degree of accuracy and speed when applied to large datasets [5]. A MDD is a generalization of the binary decision diagrams (BDD), which have been used to model static multistate systems for reliability and availability analysis. MDDs are one of the effective mathematical methods for the representation of multiple-valued logic (MVL) functions of large dimensions. They have recently been adapted for the reliability analysis of fault tolerant systems [15, 16]. The MDD can quickly categorize the degree of the risk of consequences according to the symptoms [17]. Specifically in relation to prior work, an NBC and a MDD have been widely used in many research projects and applications that require dependability, self-diagnosis, and monitoring abilities, especially in terms of fault diagnosis and monitoring systems [18]. Zhai et al. [5] proposed a method for the analysis of the multistate system (MSS) structure function by a BN and a Multistate System (MSS), which is an effective approach for the analysis and estimation of high-dimensional functions in MVL. Sharma et al. [6] developed CloudPD as the first innovative end-to-end fault management system capable of detecting, diagnosing, classifying, and suggesting remediation actions for virtualized cloud-based anomalies. Wang et al. [19] proposed EbAT, a system for anomaly identification in data centers that analyzes system metric distributions rather than individual metric thresholds. Smith et al. [20] proposed an availability model, which combines a high-level fault tree model with a number of lower level Markov models of a blade server system (IBM BladeCenter). Xu et al. [21] proposed an approach based on BNs for error diagnosis. However, a comprehensive analysis indicates that these researchers who work in different research areas use Bayes' theorem only for reasoning and for calculating the fault probability with threshold-based methods and other techniques for the detection of faults/anomalies, in which the tuning for sensitivity must be far-reaching in order to prevent an unacceptable number of false positives. Furthermore, the drawback of an extremely high number of false alarms is that it simply induces monitoring of individual metrics rather than monitoring metric combinations, which is more desirable.

Thus, in contrast to previous research and contributions, this paper addresses fault troubleshooting in cloud computing. The objective of this work is to monitor collections, develop classifiers, and analyze attributions of metrics rather than individual metric thresholds by extending the diagnosis of faults into troubleshooting while multiple measurements of decisions are still considered including priority, fault probability, risk or severity, and duration of the construction steps for fault detection and repair actions. The contribution of this paper is the following: it proposes a new theoretical approach algorithm to construct the steps of a model for fault troubleshooting by combining NBC with MDD and an ID to structure and manage fault troubleshooting on cloud anomaly detection. The theoretical approach takes into account multiple measurements (criteria), such as the priority, fault probability, risk, and duration of the repair when making high priority decisions about repair actions. The NBC is used to determine the fault probability based on the use of a Naïve-Bayes probabilistic model for fault diagnosis, whereas MDD and ID are used to subsequently combine the impact of the four measures and compute the utility value with the priority of the troubleshooting steps for each action to determine which fault is selected for repair. The decision for recovery repair is determined by the utility coefficient and the priority of the action for each repair. The case-study used in this work is the failure of a host server to start up. The theoretical proposition ensures that the most sensible action is carried out during the procedure of troubleshooting and generates the highest efficacy and cost-saving fault repair through three construction steps: (I) determining the network parameters which indicates the probability dependency relationship among all of the nodes; (II) evaluating the structure of the network topology; (III) assessing the probability of the fault being propagated, as shown in Figure 1.

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

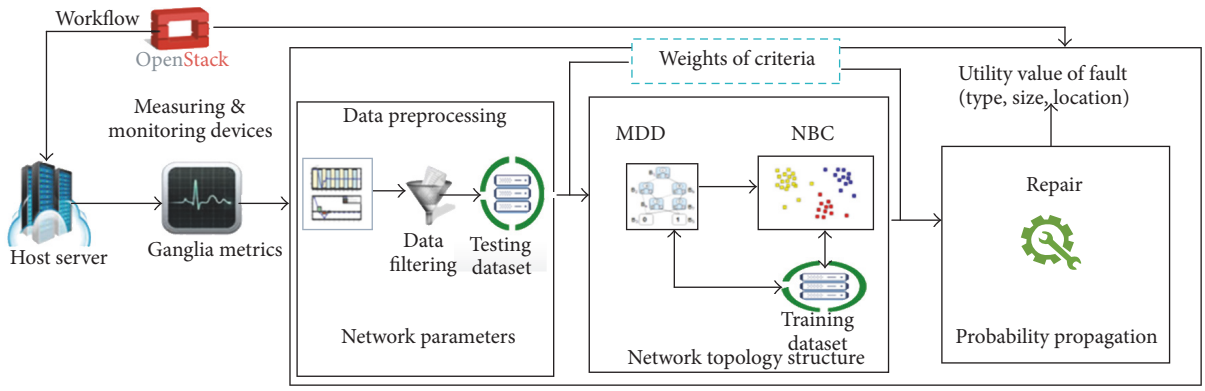


Figure 1: Overview of fault-troubleshooting workflow.

The rest of the paper is organized as follows. Section 2 describes the fault-troubleshooting analysis based on MDD, determination of the action set, and the measurements criteria. In Section 3 we introduce the concepts of the NBC and ID models. Section 4 focuses on modeling uncertainty using the proposed approach and presents our ongoing work toward developing a framework based on NBC and its extension an ID network topology and phase of parameters process. Section 5 explains probability propagation for the NBC model. Section 6 explains the process of the troubleshooting decision. Section 7 explains the experimental setup for a single data center. Section 8 presents the method implementation, evaluation results, and a comparison with current troubleshooting methods. Finally, conclusions and future work are presented in Section 9.

Table 1: Key notions for troubleshooting of faulty applications.

Questions of decision	Determine the faulty components that should be repaired in each diagnostic stage
Objectives of decision	Determine and repair faults successfully
Maker of decision	Troubleshooter with the highest priority
Measurement (criteria)	Probability fault of component Duration Severity/risk of each repair action occurring Priority services
The categories for the set of actions	The set of pairs of categories/elements (actions sets for which each process is deterministic)

2. Fault-Troubleshooting Analysis Based on MDD

A troubleshooting process aims to perform, detect, and fix faults with high efficiency and with low risk. The success of each fix is ascertained by its likelihood of failure, risk, duration, and priority. The MDD is able to rapidly categorize the degree of the risk of faults according to the symptoms [11]. MDD investigation is a procedure covering the proposal of the complication to the execution of the ultimate action. The most important notions for the application of troubleshooting for faulty applications are given in Table 1.

2.1. Ascertainment of Action Groups and Measurements Specifications. There are four categories of causes of start-up failure in a cloud system: host server, host operating system, virtual machine monitor (VMM) [22], and VMs, as shown in Figure 2. In this paper, the failure of the host server to start up is used as a case-study to show the strategy of the method. Troubleshooting for faults of the procedure is separated into two steps: the first step is to ascertain the fault group that needs to be troubleshot, and for step two which part should be troubleshot needs to be determined. The set of actions is deterministic for each of the steps. In step one, a host server start-up failure can be the result of six types of failure as shown in Figure 2, CPU utilization (CPU-utilization fault), memory (memory-leaks-fault), I/O storage (throughput-fault), network (bandwidth-fault), and other factors such as power/cooling failures; thus, the actions set in the first step includes the network, CPU, memory, storage, power, and cooling which is in need of repair. Thus, if a repair action of the CPU utilization is selected in the first step, step two will be used to select a set of actions to repair host-CPU-usage, VM-CPU-usage, and VM-CPU-Read-Time errors. As mentioned before, to perform a troubleshooting decision, troubleshooters must examine the fault probability of the fault's component and also the risk, duration, and priority required to repair the component faults. Four evaluation measurements criteria are determined: (1) probability of component failure; (2) duration; (3) risk; and (4) priority.

Fault probability indicates the likelihood of fault components causing the failure of the host server to start up. Duration is defined as the time in minutes to complete the repair fault action. Risk is defined as the degree of risk (normal, minor, and serious) of creating additional new faults and making the troubleshooter aware of the safety issue during the repair action. Priority is the priority level service of the repair actions. Of the four quantification criteria,

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

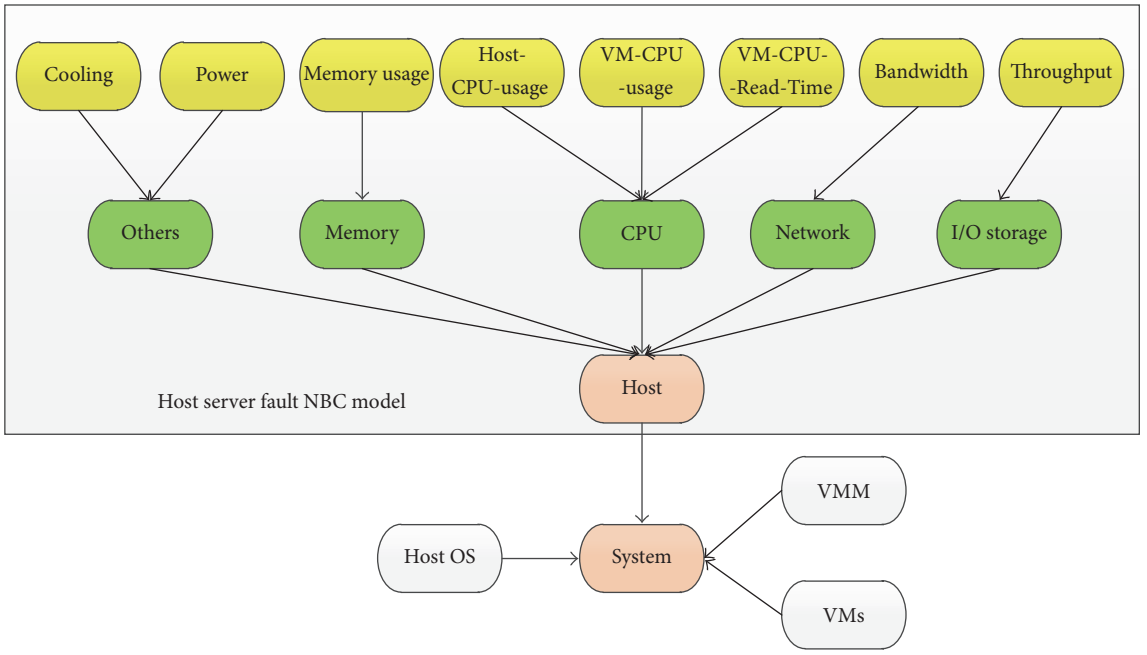


Figure 2: NBC availability model for a cloud system.

the likelihood of the fault being uncertain and the NBC diagnostic model can be used to determine such likelihood. The other three are confirmed for a particular action and are able to be inferred from expert understanding that had been accumulated over a period of time through investigation and time-consuming analysis.

2.2. Determine the Weights of Measurements. MDD assesses the effect of many measurements by computing the value of a utility in which each criterion has a specific measurement weight. The ranking method and pair-wise comparison method have been used in dictating the measurement weights. For this paper, the ranking method uses a criterion according to the importance that has been concluded to be believable by the decision makers and thus accordingly determines the weights [23]. The following formula is used for computing the weights:

$$W_c = \frac{n - r_c + 1}{\sum_{x=1}^{n} (n - r_x + 1)}, \quad (1)$$

where W_c is the weight of measurement; c, n are the decision criteria numbers; and r_c is the measurement criterion in the status of hierarchical importance of c. Based on the domain expert's opinion, the order of importance for each measurement is explained as follows:

(1) Fault probability: first

(2) Severity/risk: second

(3) Priority: third

(4) Duration: fourth.

Table 2: A sample weight of each of the measurement criteria.

Criteria	Fault probability	Risk	Priority	Duration
Weight	0.4	0.1	0.2	0.3

As seen in Table 2, a sample weight of each measurement is calculated by the ranking method.

3. Determining Fault Probability Using NBC and ID

3.1. The Naïve-Bayes Probabilistic Model. A Naïve-Bayes classifier [24] is a function that maps input feature vectors $X = \{x_1, \ldots, x_n\}$, $n \geq 1$, to output class labels $C = \{c_1, \ldots, c_n\}$, where X is the feature space. Abstractly, the probability model for a Bayes classifier is a conditional model $(C \mid x_1, \ldots, x_n)$, which is the so-called posterior probability distribution [25]. Applying Bayes' rule, the posterior can be expressed as

$$\begin{aligned} P\left(\frac{C_i}{X}\right) &= \frac{P(C_i, X)}{P(X)} = \frac{P(X|\, C_i)\, P(C_i)}{\sum_{i=1}^{n} P(X|\, C_i)\, P(C_i)} \\ &= P(x_1|\, C_i) \times P(x_2|\, C_i) \times \cdots \times P(x_n|\, C_i) \\ &\quad \times P(C_i), \quad i = 1, 2, 3, \ldots, n, \end{aligned} \quad (2)$$

where $P(C_i/X)$ is the posterior probability of the class (target) given predictor (attribute); $P(C_i)$ is the prior probability of the class; $P(X|\, C_i)$ is the likelihood probability of the given class; $P(X)$ is the prior probability of the predictor (evidence).

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The simplest classification rule is used to assign an observed feature vector X to the class with the maximum posterior probability.

$$\begin{aligned}\text{classify}(X) &= \arg\max_{c_i} P(c_i \mid X) \\ &= \arg\max_{c_i} \frac{P(X \mid c_i) P(c_i)}{\sum_{i=1}^{n} P(X \mid c_i) P(c_i)}.\end{aligned} \tag{3}$$

Because $\sum_{i=1}^{n} P(X \mid c_i)P(c_i)$ is independent of c_i and does not influence the arg max operator, classify(X) can be written as

$$\text{classify}(X) = \arg\max_{c_i} (X \mid c_i) P(c_i). \tag{4}$$

This is known as the maximum a posteriori probability (MAP) decision rule [26].

The BN model that was established for fault diagnosis contains three aspects of work [27]:

(1) Determine the network parameters to indicate the probability dependency relationship between all nodes;

(2) Determine the network topology structure;

(3) Probability propagation: this is to calculate the probability of each node given the evidence.

In this paper, the purpose of building an NBC model is to determine the most likely cause of a fault, given a fault symptom, that is, to compute the end possibilities for the cause of the fault given the evidence [28]. The end possibility is calculated using the joint probability calculator. To further simplify this calculation, the conditional independence is determined by NBC to make some assumptions. Any node is independent of unlinked nodes and is dependent on its parent node. Thus, for any node C_i, which belongs to a group of nodes, $\{C_1, C_2, \ldots, C_n\}$, if there is a clear node $\pi(C_i) \subseteq \{C_1, C_2, \ldots, C_n\}$, C_i will be conditionally independent of all other nodes except for that of $\pi(C_i)$. Conditional independent terms, according to the definition, are

$$P\left(\frac{C_i}{C_1, C_2, \ldots, C_n}\right) = P\left(\frac{C_i}{\pi(C_i)}\right). \tag{5}$$

Figure 3 is an example of the NBC model with the content arrangement of three layers with a four-node connection $\{c_1, c_2, c_3, c_4\}$. According to (2), the posterior conditional probability can be obtained by

$$P\left(\frac{C_2 = \text{True}}{C_4 = \text{True}}\right) = \frac{P(C_2 = \text{True}, C_4 = \text{True})}{P(C_4 = \text{True})}, \tag{6}$$

where $P(C_2 = \text{True}, C_4 = \text{True})$ and $P(C_4 = \text{True})$ are called marginal probability and can be calculated from

$$\begin{aligned}&P(C_2 = \text{True}, C_4 = \text{True}) \\ &\quad = \sum_{c_1, c_3} P(C_2 = \text{True}, C_1, C_3, C_4 = \text{Ture}), \\ &P(C_4 = \text{True}) = \sum_{c_1, c_2, c_3} P(C_1, C_2, C_3, C_4 = \text{Ture}),\end{aligned} \tag{7}$$

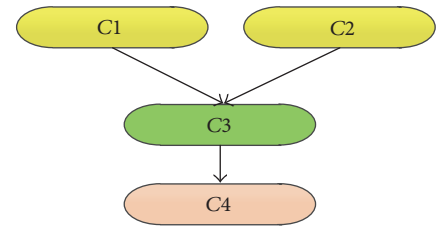


Figure 3: An example of NBC with four nodes.

where $P(C_2 = \text{True}, C_1, C_3, C_4 = \text{Ture})$ and $P(C_1, C_2, C_3, C_4 = \text{True})$ involve calculation of the joint probability. The joint probability $P(C_1, C_2, C_3, C_4)$ can be calculated according to the definition of the chain rule and joint probability, from

$$\begin{aligned}P(C_1, C_2, C_3, C_4) &= p(C_1) \prod_{i=2}^{4} P\left(\frac{C_i}{C_1, C_2, \ldots, C_{i-1}}\right) \\ &= p(C_1)\, p\left(\frac{C_2}{C_1}\right) p\left(\frac{C_3}{C_1 C_2}\right) p\left(\frac{C_4}{C_1 C_2 C_3}\right).\end{aligned} \tag{8}$$

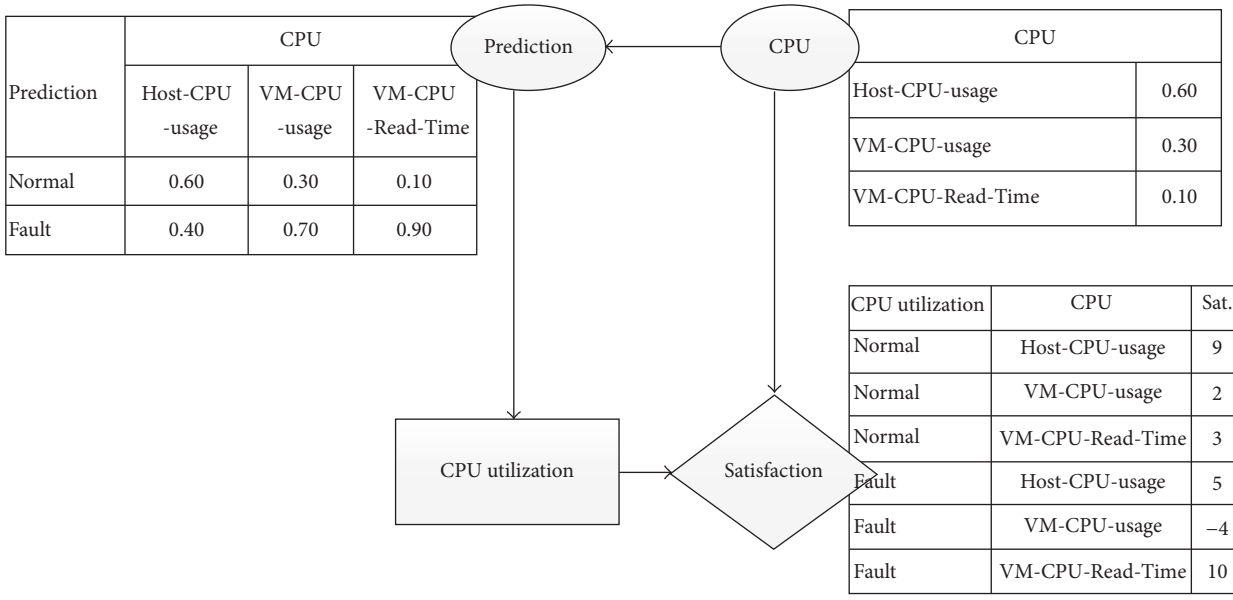
3.2. The Influence Diagrams Model. Influence diagrams (ID) [29] for solving decision problems extend BN with two additional types of nodes, utility nodes and decision nodes. A utility node is a random variable whose value is the utility of the outcome [30]. Nodes for the random variables in the BN are called chance nodes in the ID. A decision node defines the action alternatives considered by the user. A decision node is connected to those chance nodes whose probability distributions are directly affected by the decision. A utility node is a random variable whose value is the utility of the outcome. Like other random variables, a utility node holds a table of utility values for all value configurations of its parent nodes [31]. In an ID, let $A = \{a_1, a_2, \ldots, a_n\}$ be a set of mutually exclusive actions and H the set of determining variables. A utility table $U(A, H)$ is needed for yielding the utility for each configuration of action and determining variable in order to assess the actions in A. The problem is solved by calculating the action that maximizes the expected utility:

$$EU(a_i) = \sum_{H} U(a_i)\, p(H \mid a_i), \tag{9}$$

where $EU(a_i)$ represents the expected utility of action a_i; $p(H \mid a_i)$ is the conditional probability of variables h_i, where H, given action a_i, is executed. This conditional probability is calculated from the conditional probability table (CPT) while traversing the BN of these variables.

Figure 4 represents an example of an ID of a CPU and a decision to detect a CPU-utilization fault. Prediction and CPU are chance nodes containing probabilistic information about the CPU and prediction. Satisfaction is a utility or value node. CPU utilization is a decision node. The objective is to maximize expected satisfaction by appropriately selecting values of CPU utilization for each possible prediction. The values of satisfaction for each combination of CPU utilization

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



Prediction	CPU		
	Host-CPU -usage	VM-CPU -usage	VM-CPU -Read-Time
Normal	0.60	0.30	0.10
Fault	0.40	0.70	0.90

CPU	
Host-CPU-usage	0.60
VM-CPU-usage	0.30
VM-CPU-Read-Time	0.10

CPU utilization	CPU	Sat.
Normal	Host-CPU-usage	9
Normal	VM-CPU-usage	2
Normal	VM-CPU-Read-Time	3
Fault	Host-CPU-usage	5
Fault	VM-CPU-usage	−4
Fault	VM-CPU-Read-Time	10

Figure 4: Example of an influence diagram.

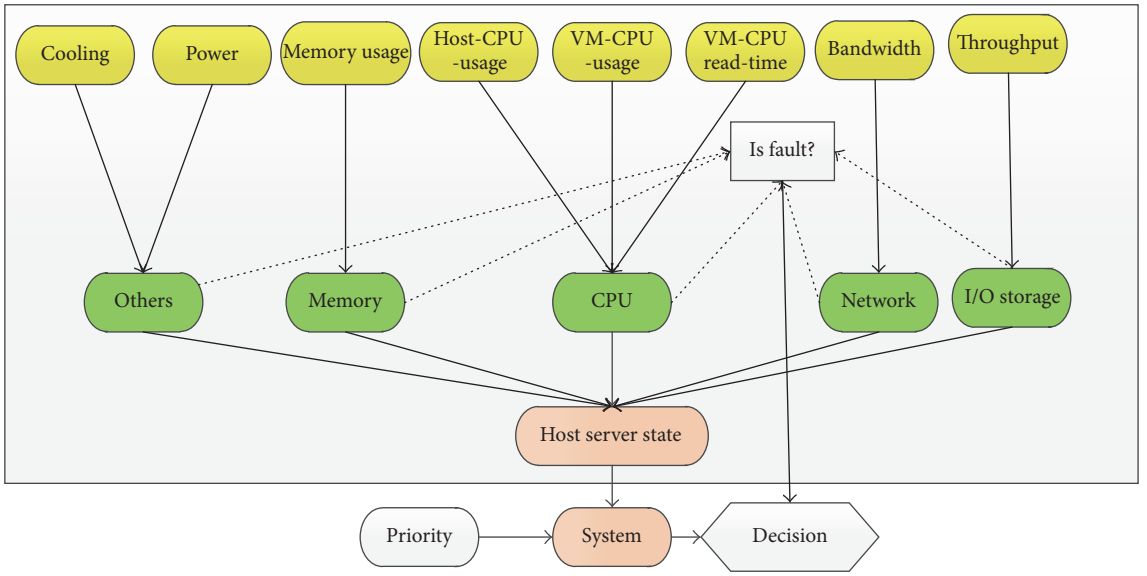


Figure 5: NBC for host server start-up failure.

and CPU are also given. The evaluation of the algorithm of an ID is performed using the following procedure [31]:

(1) Set the evidence variables for the current state.

(2) For each possible value of the decision node, set the decision node to that value.

(3) Calculate the posterior probabilities for the parent nodes of the utility node using a standard probabilistic inference algorithm.

(4) Calculate the resulting utility function for the action and return the action with the highest utility.

4. Determination of NBC Network Topology

In this section, we focus on modeling uncertainty using the proposed approach and present our ongoing work toward developing a framework based on NBC and ID models.

4.1. The Topology of the NBC Combined with an ID Model. As shown in Figure 5, start-up failure of the host server may have its roots in six groups of faults which include CPU utilization [32], memory usage, I/O storage, network overhead, and power and cooling failure [20]. Each group has some specific reasons as shown in Table 3.

Using GeNle [33] the topology of the NBC combined with an ID model was created in accordance with the causes of the

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 3: Fault reasons for the start-up failure of a host server.

Fault category	Fault causes	Measurement level
CPU utilization	% CPU time used by host CPU during normal sampling period (host-CPU-usage) % CPU time during which the CPU of the VM was actively using the physical CPU (VM-CPU-usage) % CPU time during which the CPU of the VM was ready but could not get scheduled to run on the physical CPU (VM-CPU-Ready-Time)	VM, host
Memory	% of used memory (memory usage)	VM, host
Network	% of network usage (bandwidth)	VM, host
I/O storage	% of disk usage (throughput)	Host
Others	Cooling and power	Host

Table 4: Sample parameters dataset of CPU utilization (testing/training).

Time-monitoring	VM-CPU-Ready-Time	Host-CPU-usage	VM-CPU-usage
12:02:00 AM	30.00	30.00	30.93
12:07:00 AM	30.00	35.00	78.12
12:12:00 AM	90.00	70.00	92.43
12:17:00 AM	31.84	38.21	42.19
12:22:00 AM	27.28	32.74	36.15
12:27:00 AM	32.10	38.52	42.53
12:32:00 AM	27.42	32.90	36.33
12:37:00 AM	32.10	38.52	42.53
⋮	⋮	⋮	⋮

fault of the host server start-up failure, as shown in Figure 5. The model has eight chance nodes (conditions) for the root nodes pointing to the fault root causes (attribute values), which are shown in the yellow circles, five intermediate nodes pointing to the fault category (observed attributes), which are shown as green circles, a symptom node (predict) shown as an orange circle, one node indicating the priority drawn as a white circle, one decision node (Is fault?) drawn as a white rectangle, and the utility (decision) nodes drawn as the final white shape.

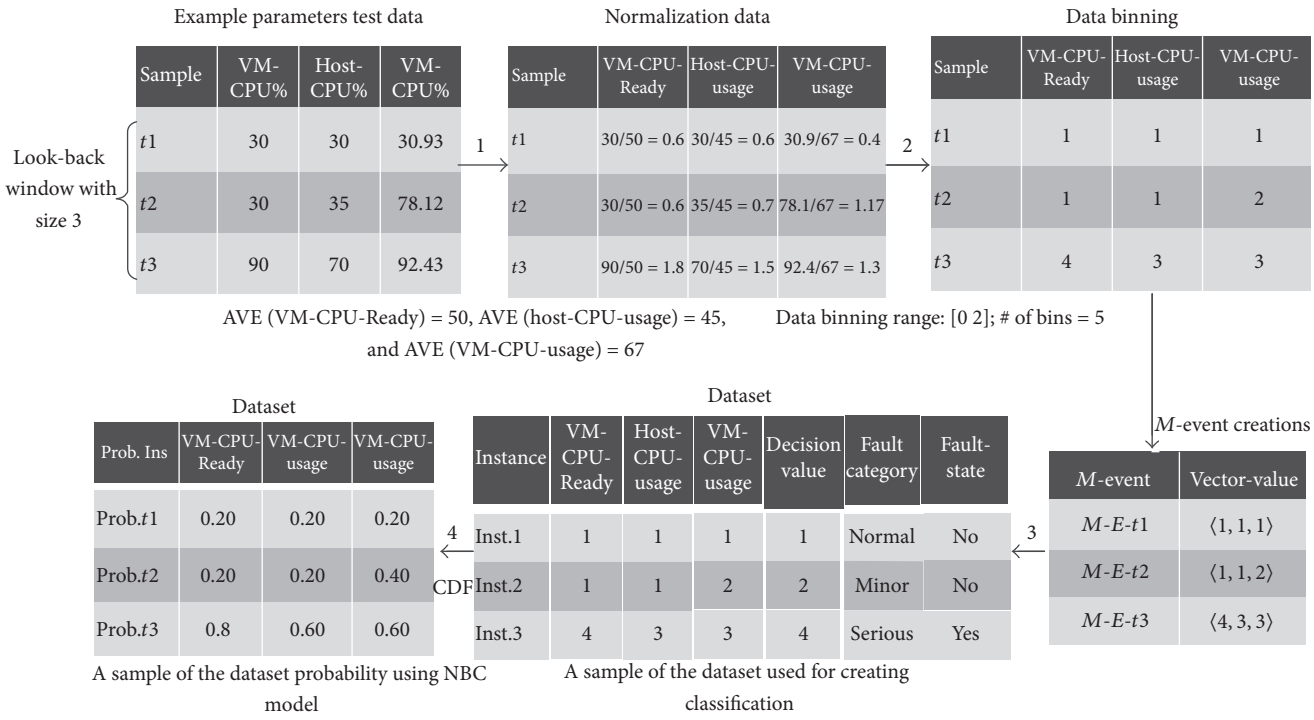
As seen in Figure 3, each node in the graph is associated with a set of local (conditional) probabilities expressing the influence of the values of the nodes' predecessors (parents) on the probabilities of the values of this node itself (e.g., $\Pr(X_i/p(X_i))$, (2)). These probabilities form the quantitative knowledge input into the NBC network. An important property of the network is (conditional) independence. The lack of a directed link between two variables from the CPU-utilization node and host-CPU-usage node are independent, conditional on some subset φ of other variables in the model. For example, if no path existed between the CPU-utilization node and host-CPU-usage then φ will be empty.

4.2. Normal Distribution. Numerical data [34] need to be transformed to their categorical counterparts (binning) before constructing their frequency tables. Other options we have are to use the distribution of the numerical variables to obtain a good estimate of the frequency. A normal probability distribution is defined by two parameters, μ and σ. Besides μ and σ, the normal probability density function (PDF) $f(x)$ also depends on the constants e and π [35]. The normal distribution is especially important as a sampling distribution for estimation and hypothesis testing.

4.3. Determination of Network Parameters. Parameters were taken from large monitoring engines (metric-collection) to obtain the metrics of interest (numerical predictors or attributes). Additional details are provided in Section 8, for example, for the Virtual Machine Manager (VMM) [36] hypervisor (testing dataset) and historical data (training dataset). Sample parameters of CPU utilization (testing/training dataset) are provided in Table 4.

The metrics of the VMs and host server were collected by running the VMs on the Xen (hypervisor) [37, 38], which was installed on the host server in combination with preprocessing (reported in Table 3) using Ganglia metrics software. Because these metrics exist in the form of numerical data, the numerical variables had to be transformed into their categorical counterparts (binning) before constructing their frequency table to use them as input for the network topology structure [39]. Therefore, as shown in Figure 6, the preprocessing step consists of four steps to translate continuous

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



Sample	VM-CPU%	Host-CPU%	VM-CPU%
$t1$	30	30	30.93
$t2$	30	35	78.12
$t3$	90	70	92.43

Sample	VM-CPU-Ready	Host-CPU-usage	VM-CPU-usage
$t1$	30/50 = 0.6	30/45 = 0.6	30.9/67 = 0.4
$t2$	30/50 = 0.6	35/45 = 0.7	78.1/67 = 1.17
$t3$	90/50 = 1.8	70/45 = 1.5	92.4/67 = 1.3

Sample	VM-CPU-Ready	Host-CPU-usage	VM-CPU-usage
$t1$	1	1	1
$t2$	1	1	2
$t3$	4	3	3

Prob. Ins	VM-CPU-Ready	VM-CPU-usage	VM-CPU-usage
Prob.$t1$	0.20	0.20	0.20
Prob.$t2$	0.20	0.20	0.40
Prob.$t3$	0.8	0.60	0.60

Instance	VM-CPU-Ready	Host-CPU-usage	VM-CPU-usage	Decision value	Fault category	Fault-state
Inst.1	1	1	1	1	Normal	No
Inst.2	1	1	2	2	Minor	No
Inst.3	4	3	3	4	Serious	Yes

M-event	Vector-value
M-E-$t1$	$\langle 1, 1, 1\rangle$
M-E-$t2$	$\langle 1, 1, 2\rangle$
M-E-$t3$	$\langle 4, 3, 3\rangle$

FIGURE 6: Explanation of the method of the proposed approach and a sample of tables from the dataset.

percentage utilization into interval probability values and generate monitoring vectors of events (M-events) by using the method proposed in [19]. We adopted this method with a filtering dataset to remove and process the outlier data and noise by using Extended Kalman Filter (EKF) [40, 41] and we generated a new method algorithm for transforming a numerical data to binning data, as shown in Algorithm 1.

Hence, the presented method has a buffer size of n (a look-back window) for the metrics of the previous n samples that had been observed (e.g., $n = 3$, range $[0, 2]$ and #Bin = 5). The look-back window is used for multiple reasons [19]: (I) shifts of work patterns may render old history data even useless or misleading, (II) at exascale, it is impractical to maintain all history data, and (III) it can be implemented in high speed RAM, which can further increase detection performance. We use a window-size of 3, sampling interval of 4 seconds, and interval length of 12 minutes. The number of data points in the 12 minutes' interval are, thus, $(12 * 60)/(3 * 4) = 60$. Once the collection of sample data is complete, the data are preprocessed and transformed into a series of bin numbers for every metric type, using (10) to perform the data binning, with a time instance server $(t_1, t_2, \ldots, t_i)$ and M-events $(M_1, M_2, \ldots, M_i)$, where i is number of instances, with the results as shown in Table 5.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i,$$
$$\sigma = \left[\frac{1}{n-1}\sum_{i=1}^{n}\left(x_i - \mu\right)^2\right]^{0.5}, \quad (10)$$
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{(-1/2)((x-\mu)/2\sigma^2)^2}.$$

In the above equations, μ is the population mean, σ is the population standard deviation, x is from the domain of measurement values $(-\infty < x < \infty)$, and n is the number of components or services. A normal probability distribution is defined by the two parameters μ and σ. Besides μ and σ, the normal probability density function (PDF) $f(x)$ depends on the constants e and π; because these attributes are numerical data, the numerical variables in Table 4 need to be transformed into their categorical counterparts (binning) before their frequency tables are constructed by (11) (e.g., look-back window-size = 3, range = $[0\ \ 2]$, and number of bins = 5), as shown in Tables 5 and 6. The values for binning-value and decision-value are determined by the following formulas:

$$\text{If } (x > 2) \text{ then binning-value} = 5, \text{ else binning-value} = \text{TRUNC}\left(\frac{x}{0.4}\right), \quad (11)$$
$$\text{decision-value} = \text{MAX}\left(\text{binning-value}\right),$$

where x is the normalization value for the attribute and 0.4 is a statistic suggested by the probability values. Then one can begin the classification and analysis into a dataset probability (predictor) by use of the cumulative distribution function (CDF) [35] as shown in Table 8. For a continuous random variable, the CDF equation is

$$P(X \le x) = \frac{x - a}{b - a}, \quad (12)$$

where a is the lower limit and b is the upper limit 5, $a \le x \le b$.

The metrics observed in the look-back window at each time instance serve as inputs for the preprocess, as shown in

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Input: Metrics_values [i, j], mean[j], m, r, n. num. //mean[j] is a list of summation columns by index i generated from step 1, calculated from Eq. (10), m and r are predetermined statistically in this experiment $r = 2$ and $m = 5$, where, r is the value range $[0, r]$, m is equal-sized bins indexed from 0 to $m - 1$. //n size of look-back window index for row table for normalized values table. //num of the number of components metrics index for column normalized values table. //Normalized [i, j] is the normalization values table. //Bin[i, j] is the a bin index data binning table. //Filtering-dataset is function algorithm to remove and process outliner and noises data.

Output: Data binning table.

```
(1)  For i = 0, i <= n; i++ do
(2)    For j = 0, j <= num, j++ do
(3)      Normalized [i, j] = Metrices_values[i, j]/mean[i]
(4)      If Normalized [i, j] > n then
(5)           Bin[i, j] = m
(6)      else
(7)           Bin[i, j]=TRANC(Normlized[i, j]/(r/m))
(8)    End for
(9)  End for
(10) Data_binning[i, j]=Filtering-dataset(Bin[i, j])
(11) End Function return (Data_binning).
```

Algorithm 1: A proposed algorithm for transforming numerical data to binning.

Table 5, so that an M-event creation is the input for every component metric for our presented methodology, as shown in Table 6.

Figure 6 shows the generated probability table of the test dataset using the CDF, which is used as input into the NBC network model. For example, a look-back window with size 3 (metrics record) creates a dataset table with the following decision-values: if (decision-value = 0 or decision-value < 2), then (fault category is "normal" and node fault-state is "no" (working)); if (decision-value = 2 or decision-value = 3), then (fault category is "minor" and node fault-state is "no" (working)); if (decision-value > 3) then (fault category is "serious" and node fault-state is "yes" (fault/anomaly), as shown in Table 7.

In the network in Figure 5, each node has two fault-states, that is, "no" (normal or working) and "yes" (fault or anomaly), and each fault category node has three states, i.e., "normal," "minor," and "serious," as shown in Table 8. In addition, each node has a probability table acquired either from monitoring engines and preprocessing (testing dataset) or from prior historical data (training dataset). The root nodes of the fault cause are expressed in terms of the previous probabilities. For other nodes, a conditional probability table is used showing the relationship of probability dependence. The probability table, as is shown in Table 9, is the "host-CPU-usage" of the root node.

Table 10 presents the conditional probability table (CPT) [42] of the CPU utilization of the fault category node. In this case, we use the bucket elimination algorithm [43, 44] to calculate the probability of each fault category such as CPU utilization according to the prior probabilities and the CPT as the following equation:

$$P(S) = P(M_1) \sum_{M_2, M_3} P(S \mid M_2, M_3) P(M_3), \quad (13)$$

Table 5: A sample dataset of data normalization used for binning with look-back window-size.

Time instance	Host-CPU-usage	VM-CPU-usage	VM-CPU-Ready-Time
t_1	0.60	0.67	0.46
t_2	0.60	0.78	1.17
t_3	1.80	1.56	1.38
t_4	1.05	1.05	1.05
t_5	0.90	0.90	0.90
t_6	1.06	1.06	1.06
t_7	0.90	1.24	0.90
t_8	1.05	1.45	1.05
t_9	1.09	1.50	1.09
⋮	⋮	⋮	⋮

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 6: A sample dataset of M-event used for diagnosis of classifier steps.

Time instance	Host-CPU-usage	VM-CPU-usage	VM-CPU-Ready-Time
t_1	1	1	1
t_2	1	1	2
t_3	4	3	3
t_4	2	2	2
t_5	2	2	2
t_6	2	2	2
t_7	2	3	2
t_8	2	3	2
t_9	2	3	2
⋮	⋮	⋮	⋮

Table 7: A sample dataset classifier with decision values.

Instance	Host-CPU-usage	VM-CPU-usage	VM-CPU-Ready- Time	Decision value	Fault category	Fault-state
Inst.$_1$	1	1	1	1	Normal	No
Inst.$_2$	1	1	2	2	Minor	No
Inst.$_3$	4	3	3	4	Serious	Yes
Inst.$_4$	2	2	2	2	Minor	No
Inst.$_5$	2	2	2	2	Minor	No
Inst.$_6$	2	2	2	2	Minor	No
Inst.$_7$	2	3	2	3	Minor	No
Inst.$_8$	2	3	2	3	Minor	No
Inst.$_9$	2	3	2	3	Minor	No
⋮	⋮	⋮	⋮	⋮	⋮	⋮

Table 8: A sample dataset probability (predictor) using in proposed approach methodology.

Process instances	Host-CPU-usage	VM-CPU-usage	VM-CPU-Ready-Time	Fault category	Fault-state
Process.inst.$_1$	0.20	0.20	0.20	Normal	No
Process.inst.$_2$	0.20	0.20	0.40	Minor	No
Process.inst.$_3$	0.80	0.60	0.60	Serious	Yes
Process.inst.$_4$	0.40	0.40	0.40	Minor	No
Process.inst.$_5$	0.40	0.40	0.40	Minor	No
Process.inst.$_6$	0.40	0.40	0.60	Minor	No
Process.inst.$_7$	0.50	0.75	0.50	Minor	No
Process.inst.$_8$	0.50	0.75	0.50	Minor	No
Process.inst.$_9$	0.50	0.75	0.50	Minor	No
⋮	⋮	⋮	⋮	⋮	⋮

Table 9: The host-CPU-usage root node (prior probability).

State	Host-CPU-usage
No	0.12
Yes	0.88

where S is the CPU utilization and M_1, M_2, and M_3 represent the VM-CPU-Ready-Time, host-CPU-usage, and VM-CPU-usage probability values, respectively.

S subjective estimates from experts and learning from case data are the methods by which the table can be created. Example of numerical predictors is sample numerical values for host-CPU-utilization predictors as shown in Table 11.

Assume we have new evidence for prediction, such as $P(\text{Host_CPU_utilization} = 61.34 \mid \text{Yes}, \text{No})$. What is the prediction of the fault-state for this evidence? Solution is as follows: we normalize the numerical values = 61.34 and obtain the maximum value of the normal distribution function values, as follows:

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 10: CPT of CPU-utilization fault node.

VM-CPU-Ready-Time	No				Yes			
Host-CPU-usage	No		Yes		No		Yes	
VM-CPU-usage	No	Yes	No	Yes	No	Yes	No	Yes
Normal	0.17	0.23	0.11	0.12	0.19	0.13	0.21	0.11
Minor	0.52	0.13	0.32	0.25	0.14	0.15	0.14	0.31
Serious	0.31	0.64	0.58	0.63	0.67	0.72	0.65	0.58

Table 11: Sample host-CPU-utilization numerical values.

		Host-CPU-utilization					Mean (μ)	St-Dev (σ)
Fault-state	Yes	30.00	30.00	90.00	31.84	27.28	41.82	26.98
	No	57.56	41.10	41.10	36.80	40.42	43.39	8.11

$$
\begin{aligned}
&(1)\ P\left((\text{Host_CPU_utilization} = 61.34) \mid (\text{Fault_state} = \text{Yes})\right)\\
&\quad = f\left(61.34\right) = \frac{1}{26.98\sqrt{2\left(3.14\right)}} e^{(-1/2)((61.34-41.82)/2(26.98)^2)^2}\\
&\quad = 0.0058,\\
&(2)\ P\left((\text{Host_CPU_utilization} = 61.34) \mid (\text{Fault_state} = \text{No})\right)\\
&\quad = f\left(61.34\right) = \frac{1}{8.11\sqrt{2\left(3.14\right)}} e^{(-1/2)((61.34-43.36)/2(8.11)^2)^2}\\
&\quad = 0.0487.
\end{aligned}
\tag{14}
$$

Result. Max(P((Host_CPU_utilization = 61.34)|(Fault_state = Yes)), P((Host_CPU_utilization = 61.34) | (Fault_state = No))) = Max (0.0058, 0.0487) = 0. 0487. Then, fault-state = no.

5. Probability Propagation

NBC can propagate the probability for the given evidence after the structure of the model and the establishment of the CPT of all nodes. The NBC model calculates the probability and exerts inference when the host server is set to "fault" and cannot start properly for each node. The results of the probability propagation of the NBC model are shown in Figure 7. As illustrated in Figure 7, the fault probability of CPU utilization in the fault categories is the highest at 74% and is shown in Figure 7. Within this category the fault cause host-CPU-usage has the highest fault probability of 88%. Likewise, when new evidence like a test result is input into the model, the model will update all of the other nodes with respect to their new probabilities.

6. Processing Troubleshooting Decision

A troubleshooting decision involves shaping the action of repair, which is to be done in every fault-troubleshooting step. The MDD analytical method is used to calculate the values of utility and assorted actions of priority values, which are used to integrate the state of the five measurements. The action with the highest value of utility will be selected for services detection and repair. One of the MDD evaluation methods, namely, the MVL utility approach [45, 46], is adopted in this paper. Each criterion is assumed to be measurable on a ratio scale. The value of a criterion M_i can be normalized to a scale of $[0\ \ 1]$, where 0 and 1 represent the "worst" and "best" effect for the measurement criteria, respectively. The criterion risk, for example, is measured by the probability of severity $[0\ \ 1]$. An action with a severity equal to 0.65 has a higher severity than an action with a severity equal to 0.85. The criterion priority, which is given in seconds, can also be normalized to $[0\ \ 1]$. An action with a priority equal to 0.25 takes a higher priority to service than one with a priority equal to 0.5. As seen in Table 12, the five measurements are given values mapped to the range $[0\ \ 1]$ in step one to enable a decision to be made regarding which fault group needs fixing. For example, the "best" value for the severity of the five identified repair actions is 0.85 (fault-repair-others). Likewise, the "best" value for duration is 0.75 (fault-repair-CPU). The indistinct values for the measurement criteria are found using NBC.

MDD evaluates the weight of influence for multiple criteria by calculating the value of utility whereby all of the criteria have their own individual weights. All criterions are individually given a weight u_i that indicates how important the measurement criterion is, and the whole of the utility $U(a)$ of an action a is the weighted sum.

$$U\left(a\right) = \sum u_i M_i\left(a\right). \tag{15}$$

Table 13 lists the calculated utility values of five actions in the first step of the decision-making process. The troubleshooters should proceed to the CPU fault in the first step because it has a high utility value 0.69, as shown in Table 13. Because the decision to repair the CPU fault was made in step 1, in the second troubleshooting step the set of actions repairing the host-CPU-usage, VM-CPU-usage, and the VM-CPU-Ready-Time are included. The values normalized to $[0\ \ 1]$, based on the three criteria, for every action in step one are presented in Table 14.

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

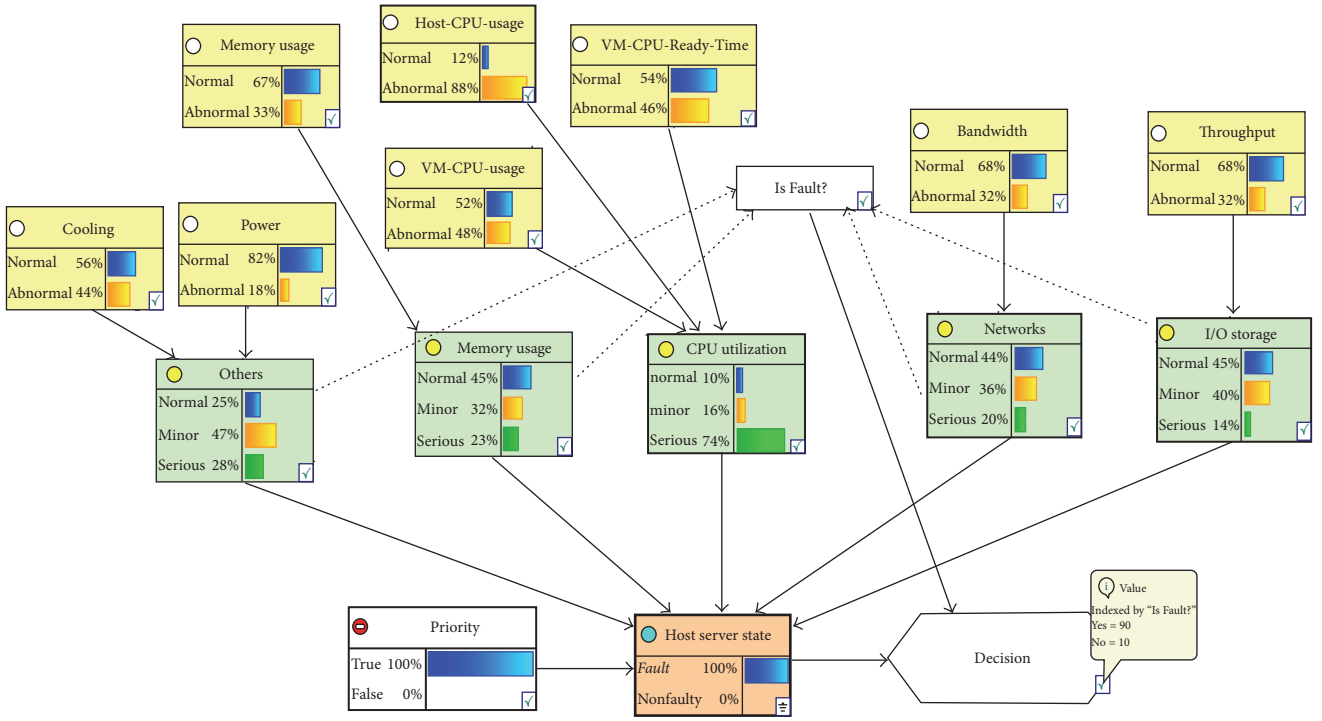


Figure 7: Probability propagation of the NBC model.

Table 12: Measurement values mapped to the range of a [0 1] scale in the first step process.

Measurement (criteria)	Severity/risk	Duration	Priority	Fault probability
Fault-repair-CPU	0.65	0.75	0.25	0.85
Fault-repair-memory	0.75	0.52	0.35	0.76
Fault-repair -network	0.55	0.45	0.45	0.86
Fault-repair-I/O-storage	0.50	0.40	0.50	0.55
Fault-repair-others	0.85	0.65	0.50	0.70

Table 13: Lists the calculated utility values of five actions in the process in the first step.

Measurement (criteria)	Severity/risk $w = 0.1$	Duration $w = 0.3$	Priority $w = 0.2$	Fault probability $w = 0.4$	Utility value
Fault-repair-CPU	0.07	0.23	0.05	0.34	**0.69**
Fault-repair-memory	0.08	0.16	0.07	0.30	0.61
Fault-repair-network	0.06	0.14	0.09	0.34	0.63
Fault-repair-storage	0.05	0.12	0.10	0.22	0.49
Fault-repair-others	0.09	0.20	0.10	0.28	0.67

Table 15 contains the calculated efficacy values of the decision-making procedure. As seen in Table 15, the utility value of 0.70 for the action of repairing host-CPU-usage is the highest. Thus, the troubleshooting decision is to repair the host-CPU-usage fault of the host server.

7. Experimental Setup

The architecture of the physical machine that was used as experimental setup is shown in Figure 8. The test bed uses two virtual machines VM1 and VM2 configured on a Xen (hypervisor) platform hosted on one Dell blade server with dual core 3.2 GHz CPUs and 8 GB RAM.

OpenStack is open source software capable of controlling large pools of storage, computing, and networking resources throughout a data center, managed through a dashboard or via the OpenStack API [47]. We injected 40 anomalies into the OpenStack online service on the host server, which results in faults or anomalies for global resource consumption, as presented in Figure 9. These 40 anomalies stem from extreme failure source issues in online services [19, 48]. The following patterns of CPU utilization were observed when the test bed

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

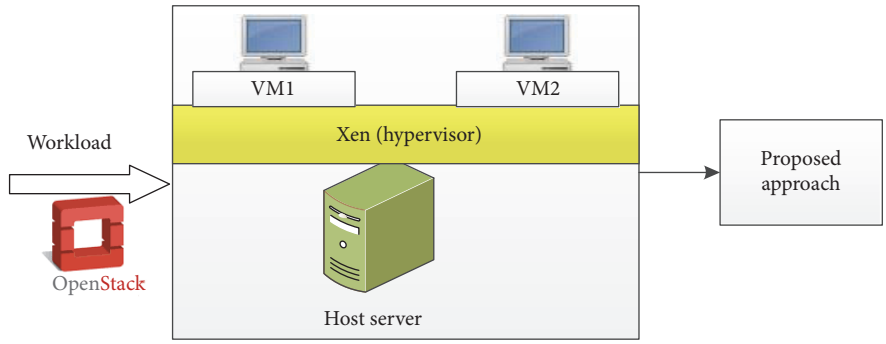


Figure 8: Architecture of the data center employed for the experimental setup.

Table 14: The values mapped to [0 1] weigh based on the three criteria in the process of the first step.

Measurement (criteria)	Severity/risk	Duration	Priority	Fault probability
Repair Host-CPU-usage	0.60	0.80	0.10	0.95
Repair VM-CPU-usage	0.50	0.70	0.2	0.43
Repair VM-CPU-Ready-Time	0.55	0.45	0.30	0.50

Table 15: Weighted utility values of actions in the second step

Measurement (criteria)	Severity/risk $w = 0.1$	Duration $w = 0.3$	Priority $w = 0.2$	Fault probability $w = 0.4$	Utility value
Repair host-CPU-usage	0.06	0.24	0.02	0.38	**0.70**
Repair VM-CPU-usage	0.05	0.21	0.04	0.17	0.47
Repair VM-CPU-Ready-Time	0.06	0.14	0.06	0.20	0.46

exhibited the expected behavior for the host server, that is, the case-study in this work. The metrics of the both the VMs and the host were collected using Ganglia metrics and analyzed according to a fault-troubleshooting approach from a previous study, for four seconds. During this period, we were injecting the anomalies into the testbed system; more details for implementation are discussed in Section 8.

8. Approach Method Implementation and Evaluation Results

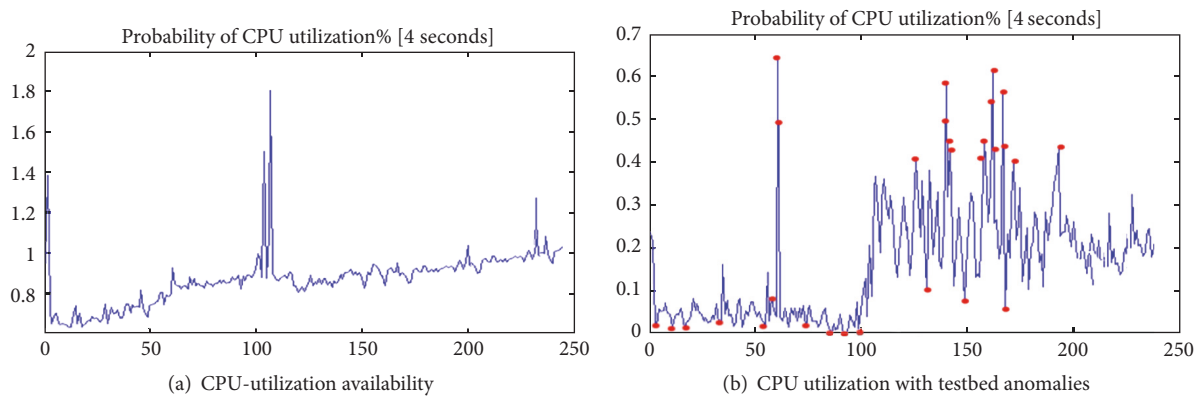
We implement our approach at both the VM level and host server level. The host server is considered as the direct parent of the VMs. Thus, the local time series is calculated first for the VMs, followed by their aggregation for the host server's global time series. As mentioned in Section 4, the first implementation uses the Ganglia metrics method to record and to identify global resource provisioning anomalies. Forty anomaly samples were injected into the testbed, leading to global resource consumption by the faults/anomalies, which does not exclude the CPU utilization of the running host server. As shown in Figure 10, the aggregation of the anomalies is conducted by integrating the measurements performed with the Graphite tool with the Ganglia metrics to obtain the CPU-utilization metrics (host-CPU-usage, VM-CPU-usage, and VM-CPU-Read-Time). The identification of a fault/anomaly is performed by applying a threshold-based methodology and troubleshooting approach for the online CPU-utilization observations of the VMs and the host server.

As seen in Figure 10, the violations are indicated by red dots when the baseline threshold method is applied. The consecutive red dots are considered as a single anomaly as there may be a delay period of some time. A positive alarm and its matching actual faulty/anomaly injection into the testbed may occur at different times because of the delay between the injection and its effects based on the metrics being collected and analyzed. In the presented paper, such a problem needs to be developed. To set the threshold values optimally, 5% and 90% are chosen as the lower and upper probability boundaries, respectively, for the thresholds based on representative values used in state-of-the-art deployments. As shown in Figure 11, a similar implementation is used for the host server to obtain CPU-utilization data utilizing these threshold values to identify faults/anomalies. The anomaly events are represented by dotted lines and red dots indicate the position of an alarm.

Four statistical measures [6] are used to evaluate the effectiveness of the construction steps of the troubleshooting procedure to detect faults/anomalies in the testbed of our experimental setup which is explained in Section 7, as shown in Table 16.

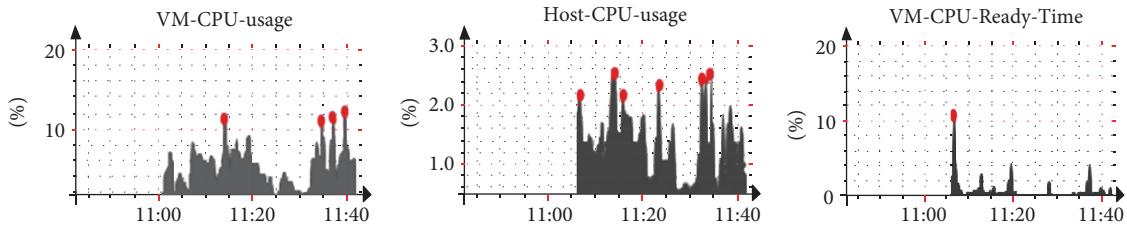
Overall, as compared with the method of threshold-based detection, the results obtained show a 94% improvement in the accuracy (*F1*) score, on average, against the theoretical

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



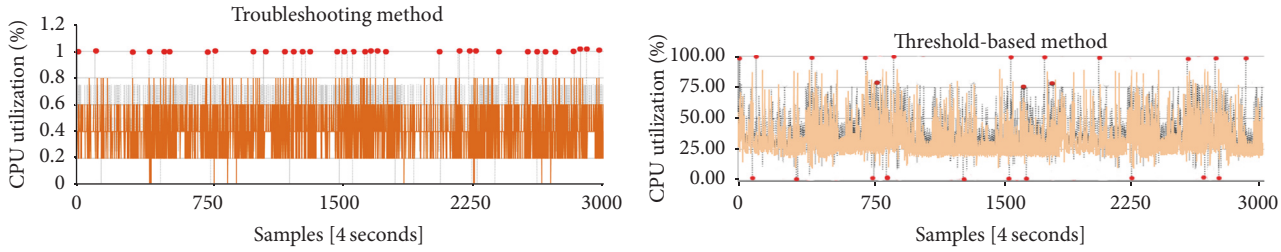
(a) CPU-utilization availability (b) CPU utilization with testbed anomalies

Figure 9: CPU parameters using Ganglia monitoring metrics.



(a) VM-CPU-usage (b) Host-CPU-usage (c) VM-CPU-Read-Time

Figure 10: CPU-utilization metrics using Ganglia and Graphite for the OpenStack application running on the host server.



(a) Troubleshooting method trace (b) Threshold-based method trace

Figure 11: A similar implementation for the host server to obtain CPU utilization.

approach, with an average false-alarm rate of 0.03% as shown in Table 17.

9. Conclusion

Fault troubleshooting for cloud computing has the goal of diagnosing and repairing faults at the highest level of effectiveness and accuracy and with minimal risk. The effectiveness is dependent upon multiple measurement criteria including the fault probability, priority, risk, and duration of how long the repair action takes. In this paper, a new fault-troubleshooting decision method algorithm is proposed based on NBC and MDD, which includes an influence diagram. The method makes certain that the most sensible repair action is chosen for each of the troubleshooting steps, thus enabling rapid, highly efficient, and low-risk troubleshooting. The practical consideration for implementing this approach is to provide a decision-theoretical methodology for modeling the construction steps for fault troubleshooting of cloud computing. The failure of a host server to start up is utilized as a case-study in this paper, although the proposed method has more robust significance for troubleshooting cloud platforms. In conclusion, the proposed method contains the following construction steps: (1) identification of possible actions for the set; (2) identification of measurement criteria for the set that are attributes of actions in which available options are determined; (3) establishing the uncertainty measurement criteria such as fault potential and certain measurements such as duration, risk, and fault probability; (4) using the NBC and the MDD with ID to build a model to surmise the ambiguity measurements criteria; (5) for each criterion determining the values and weights and then normalizing the criterion values to a scale of $[0\ \ 1]$; establish the general utility values of the actions employing the weighted sum method; (6) using the utility and priority values acquired, generating a decision. A high utility value indicates a high service priority.

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 16: Four measures in statistics.

Precision	Recall	Accuracy	False-alarm rate (FAR)
$\frac{\text{successfuldetections}}{\text{oftotalalarms}}$	$\frac{\text{successfuldetections}}{\text{oftotalanomalies}}$	$\frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$	1 − presision

Table 17: Experiment results.

Methods	Description	Alarms	Successful detection instances	Recall	Precision	Accuracy (F_1)	FAR
A proposed approach	Troubleshooting model	38	37	0.92	0.97	**0.94**	**0.03**
Threshold-based method	Static thresholds: 5% > static thresholds > 90%	16	14	0.35	0.87	0.49	0.13

Future work regarding the proposed approach includes the following: (I) evaluating scalability with multiple virtual machines, (II) evaluation of scalability for large datasets based on the Hadoop MapReduce and Apache Spark platforms for analyzing large volumes of real-time datasets by using aggregation, (III) additionally, the correction and recovery of each fault are expected to lead to improvement with the use of new reasoning algorithms, such as case-based reasoning (CBR).

Conflicts of Interest

The authors declare that they have no conflicts of interest.

Acknowledgments

This work was supported by the Chinese High Tech R&D (863) Program Project "Cloud Computer Test and Evaluation System Development (2013AA01A215)."

References

[1] J. S. Ward and A. Barker, "Cloud cover: monitoring large-scale clouds with Varanus," *Journal of Cloud Computing*, vol. 4, no. 1, article 16, 2015.

[2] I. Nwobodo, "Cloud computing: Models, services, utility, advantages, security issues, and prototype," *Lecture Notes in Electrical Engineering*, vol. 348, pp. 1207–1222, 2016.

[3] L. Bi, Y. Cheng, D. Hu, and W. Lv, "Intelligent Diagnostics Applied Technology of Specialized Vehicle Based on Knowledge Reasoning," in *Proceedings of the 4th International Conference on Computer Engineering and Networks*, pp. 415–424, Springer, 2015.

[4] T. Suzuki, T. Kimura, K. Hirata, and M. Muraguchi, "Adaptive virtual machine assignment for multi-tenant data center networks," in *Proceedings of the International Conference on Computer, Information and Telecommunication Systems, IEEE CITS 2015*, July 2015.

[5] S. Zhai, S. Sheng, and S. Z. Lin, "Bayesian networks application in multi-state system reliability analysis," *Applied Mechanics and Materials*, vol. 347, pp. 2590–2595, 2013.

[6] B. Sharma, P. Jayachandran, A. Verma, and C. R. Das, "CloudPD: Problem determination and diagnosis in shared dynamic clouds," in *Proceedings of the 43rd Annual IEEE/IFIP International Conference*, pp. 1–12, Jun 2013.

[7] N. Guarracino, V. Lavorini, A. Tarasio, and E. Tassi, "An Integrated Monitoring System, with Ganglia and Nagios, for the Cosenza ReCaS Site , High Performance Scientific Computing Using Distributed Infrastructures," in *Results and Scientific Applications Derived from the Italian Pon Recas Project*, 2016.

[8] M. L. Massie, B. N. Chun, and D. E. Culler, "The ganglia distributed monitoring system: design, implementation, and experience," *Parallel Computing*, vol. 30, no. 7, pp. 817–840, 2004.

[9] J. Kostolny, E. Zaitseva, S. Stojković, and R. Stanković, "Application of multi-valued decision diagrams in computing the direct partial logic derivatives," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics): Preface*, vol. 9520, pp. 41–48, 2015.

[10] E. Zaitseva and V. Levashenko, "Decision diagrams for reliability analysis of multi-state system," in *Proceedings of the International Conference on Dependability of Computer Systems, DepCoS - RELCOMEX 2008*, pp. 55–62, June 2008.

[11] A. Alkasem, H. Liu, Z. Decheng, and Y. Zhao, "AFDI: a virtualization-based accelerated fault diagnosis innovation for high availability computing," in *Software Engineering*, pp. 1–8, 2015.

[12] X. Miao, Y. Gao, G. Chen, B. Zheng, and H. Cui, "Processing incomplete k nearest neighbor search," *IEEE Transactions on Fuzzy Systems*, vol. 24, no. 6, pp. 1349–1363, 2016.

[13] S. Sajja and L. A. Deleris, "Bayesian network structure learning with messy inputs: the case of multiple incomplete datasets and expert opinions," in *Algorithmic decision theory*, vol. 9346 of *Lecture Notes in Comput. Sci.*, pp. 123–138, Springer, 2015.

[14] E. M. Vieira, J. M. Silva, and L. B. Silva, "Modeling Bayesian Networks from a conceptual framework for occupational risk analysis," *Production*, vol. 27, pp. 1–12, 2017.

[15] Y. Mo, L. Xing, L. Cui, and S. Si, "MDD-based performability analysis of multi-state linear consecutive-k-out-of-n: F systems," *Reliability Engineering & System Safety*, vol. 166, pp. 124–131, 2017.

[16] Q. Zhai, L. Xing, R. Peng, and J. Yang, "Multi-Valued Decision Diagram-Based Reliability Analysis of *k*-out-of-*n* Cold Standby Systems Subject to Scheduled Backups," *IEEE Transactions on Reliability*, vol. 64, no. 4, pp. 1310–1324, 2015.

[17] Y. Dai, Y. Xiang, and G. Zhang, "Self-healing and hybrid diagnosis in cloud computing," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics): Preface*, vol. 5931, pp. 45–56, 2009.

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[18] A. Alkasem, H. Liu, and D. Zuo, "Utility Cloud: A Novel Approach for Diagnosis and Self-healing Based on the Uncertainty in Anomalous Metrics," in *Proceedings of the 2017 International Conference on Management Engineering, Software Engineering and Service Sciences*, pp. 99–107, ACM, Wuhan, China, January 2017.

[19] C. Wang, V. Talwar, K. Schwan, and P. Ranganathan, "Online detection of utility cloud anomalies using metric distributions," in *Proceedings of the 12th IEEE/IFIP Network Operations and Management Symposium, NOMS 2010*, pp. 96–103, IEEE, April 2010.

[20] W. E. Smith, K. S. Trivedi, L. A. Tomek, and J. Ackaret, "Availability analysis of blade server systems," *IBM Systems Journal*, vol. 47, no. 4, pp. 621–640, 2008.

[21] X. Xu, L. Zhu, D. Sun et al., "Error Diagnosis of Cloud Application Operation Using Bayesian Networks and Online Optimisation," in *Proceedings of the 11th European Dependable Computing Conference, EDCC 2015*, pp. 37–48, IEEE, September 2015.

[22] X.-Y. Ren and Y.-Q. Zhou, "A review of virtual machine attack based on Xen," in *Proceedings of the International Seminar on Applied Physics, Optoelectronics and Photonics, APOP 2016*, May 2016.

[23] Y. Hairu, J. Zhengyi, and Z. Haiyan, "Exploring a method for ranking objects based on pair wise comparison," in *Proceedings of the IEEE Fourth International Joint Conference*, pp. 382–386, Apr 2011.

[24] M. N. Murty and V. S. Devi, *Pattern Recognition: An Algorithmic Approach*, Springer Science & Business Media, 2011.

[25] S. Conrady and L. Jouffe, *Bayesian Networks and BayesiaLab: A Practical Introduction for Researchers*, Bayesia, 2015.

[26] S. Z. Li, "Markov random field models in computer vision," in *Proceedings of the European Conference on Computer Vision*, pp. 361–370, Springer, Berlin, Germany, 1994.

[27] C. C. Aggarwal, *Data Classification: Algorithms and Applications*, CRC Press, New York, NY, USA, 2014.

[28] W. Watthayu and Y. Peng, "A Bayesian network based framework for multi-criteria decision making," in *Proceedings of the 17th international conference on multiple criteria decision analysis, British Columbia CA: Whistler*, pp. 6–11, Aug 2004.

[29] A. Janjić, M. Stanković, and L. Velimirović, "Multi-criteria Influence Diagrams – A Tool for the Sequential Group Risk Assessment," in *Granular Computing and Decision-Making*, vol. 10 of *Studies in Big Data*, pp. 165–193, Springer International Publishing, 2015.

[30] G. Deodatis, B. Ellingwood, and D. Frangopol, *Safety, Reliability, Risk and Life-Cycle Performance of Structures and Infrastructures*, CRC Press, 2014.

[31] R. A. Brooks, "Intelligence without representation," *Artificial Intelligence*, vol. 47, no. 1–3, pp. 139–159, 1991.

[32] Vcpguy, "CPU performance monitoring on ESX and VMs," 2010, http://vmwaredevotee.com/2010/10/11/cpu-performance-monitoring-on-esx-and-vms/.

[33] T. Loboda and M. Voortman, GeNie & Smile, 2017, https://www.bayesfusion.com/.

[34] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*, Texts in Statistical Science Series, CRC Press, Boca Raton, FL, USA, 3rd edition, 2014.

[35] D. P Doane and L. E. Seward, *Applied statistics in business and economics*, Irwin, 2005.

[36] B. Posey, "Using virtual machine manager as a cross-platform management tool," in *Hyper-V for VMware Administrators*, pp. 107–154, 2015.

[37] P. Barham, B. Dragovic, K. Fraser et al., "Xen and the art of virtualization," in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pp. 164–177, New York, NY, USA, October 2003.

[38] A. R. Riddle and S. M. Chung, "A survey on the security of hypervisors in cloud computing," in *Proceedings of the 2015 35th IEEE International Conference on Distributed Computing Systems Workshops, ICDCSW 2015*, pp. 100–104, July 2015.

[39] S. Sayad, *Real time data mining. Canada*, Self-Help Publishers, 2011.

[40] G. F. Welch, "Kalman filter," in *Computer vision*, pp. 435–437, Springer, 2014.

[41] K. Fujii, "Extended kalman filte," in *Reference Manual*, 2013, Extended kalman filter.

[42] F. V. Jensen, *An introduction to Bayesian networks*, vol. 210, UCL press, 1996.

[43] J. E. Gallardo, C. Cotta, and A. J. Fernández, "A Memetic Algorithm with Bucket Elimination for the Still Life Problem," in *Evolutionary Computation in Combinatorial Optimization*, vol. 3906 of *Lecture Notes in Computer Science*, pp. 73–85, Springer, Berlin, Germany, 2006.

[44] R. Dechter, "Bucket elimination: a unifying framework for reasoning," *Artificial Intelligence*, vol. 113, no. 1-2, pp. 41–85, 1999.

[45] , *Computer science and multiple-valued logic: theory and applications*, D. C. Rine, Ed., Elsevier, 2014.

[46] E. Zaitseva and V. Levashenko, "Multiple-valued logic mathematical approaches for multi-state system reliability analysis," *Journal of Applied Logic*, vol. 11, no. 3, pp. 350–362, 2013.

[47] O. Sefraoui, M. Aissaoui, and M. Eleuldj, "OpenStack: toward an open-source solution for cloud computing," *International Journal of Computer Applications*, vol. 55, no. 3, pp. 38–42, 2012.

[48] P. Soila and P. Narasimhan, "Causes of Failure in Web Applications," Tech. Rep., 2005.

9071, 2017, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/2017/9038634 by Capes, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License